

Batch – T5**Assignment No. – 5****Student Name – Sharaneshwar Bharat Punjal****Student PRN – 23520011**

1) You are working on the city construction project. You have A houses in the city. You have to divide these houses into B localities such that every locality has at least one house. Also, every house in a locality should have a telephone connection wire with each of the other houses in the locality. You are given integers A and B. **Task** – Print the minimum and the maximum number of telephone connections possible if you design the city accordingly.

Algorithm:

1. Compute the maximum possible locality size, and calculate the maximum number of connections using combinatorics.
2. Calculate the quotient and remainder when dividing the houses among localities.
3. Use the quotient and remainder to determine the minimum number of connections.
4. Return the calculated minimum and maximum connections.

Code:

```
import java.util.Scanner;

public class Q1 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        // Input values for n and m
        long n = sc.nextLong();
        long m = sc.nextLong();

        long kmin, kmax;
        long x;

        x = n / m + 1;
        kmin = ((x * (x - 1)) / 2) * (n % m);
        x -= 1;
        kmin += ((x * (x - 1)) / 2) * (m - (n % m));

        // Calculating kmax
        x = n - m + 1;
        kmax = (x * (x - 1)) / 2;

        // Output the results
        System.out.println(kmin + " " + kmax);

        sc.close();
    }
}
```

Test Case 1:

6 2

Output:

6 10

Test Case 2:

5 1

Output:

10 10

Test Case 3:

247 85

Output:

239 13203

Test Case 4:

522 112

Output:

968 84255

2) You are working in the Data Consistency team of your company. You are allocated a task as follows:

You have a data stream consisting of an equal number of odd and even numbers. You can make separations in the data stream but the number of odd elements should be equal to the number of even elements in both partitions after separation. Also, if you make a separation between a number x and number y , then the cost of this operation will be $|x-y|$ coins.

You are given the following:

An integer N

An array arr

An integer K

Task - Determine the maximum number of separations that can be made in the array by spending no more than K coins.

Algorithm:

1. Traverse the array and count odd and even numbers up to each index.
2. Whenever the number of odd and even numbers are equal, calculate the separation cost between the current and next element.
3. Sort the separation costs in ascending order to minimize the total cost.
4. Keep adding the costs to the total until the sum exceeds the allowed budget, K .
5. Return the maximum number of separations you can make within the given budget

Code:

```
import java.util.*;

public class Q2 {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
```

```
int K = sc.nextInt();
int[] arr = new int[N];
for (int i = 0; i < N; i++) {
    arr[i] = sc.nextInt();
}

int result = Max(N, K, arr);
System.out.println(result);

sc.close();
}
```

```
static int Max(int N, int K, int[] arr) {
    int result = 0;
    int even = 0, odd = 0;
```

```
// Create a list to store the differences when even and odd counts match
List<Integer> list = new ArrayList<>();
```

```
// Loop through the array to find even and odd counts
```

```
for (int i = 0; i < N - 1; i++) {
    if (arr[i] % 2 == 0) even++;
    else odd++;
```

```
// When even and odd counts are equal, add the absolute difference to the
```

list

```
if (even == odd) {
    list.add(Math.abs(arr[i + 1] - arr[i]));
}
}
```

```
Collections.sort(list);
```

```
// Calculate the total cost and increment result based on the budget (K)
```

```
int total = 0;
for (int diff : list) {
    total += diff;
    if (total <= K) result++;
    else break;
}
```

```
return result;
```

```
}
```

```
}
```

Test Case 1:

4

10

1 3 2 4

Output:

0

Test Case 2:

6
3
1 2 3 4 5 6

Output:

2

Test Case 3:

10
15
5 7 9 8 6 4 2 1 3 5

Output:

2

Test Case 4:

25
50
1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25

Output:

12

3) Bob has an array A of size N, and he is very fond of two integers X and Y. Find the length of the longest subarray, such that it contains exactly X distinct integers and Y exist at least once in the subarray.

Input Format -

The first line contains an integer T, which denotes the number of test cases.

The first line of each test case contains three space separated integers N, X, Y denoting the size of array A, the value of X and Y, respectively.

The second line of each test case contains N space-separated integers, denoting the elements of array A.

Output Format -

For each test case, print the length of the largest subarray, such that it contains exactly X distinct integers and Y exist at least once in the subarray in a new line.

Algorithm:

1. Use a sliding window approach, expanding the window by moving the right pointer and adding the current element to a frequency map.
2. Maintain the count of distinct elements within the window.
3. If the number of distinct elements exceeds X, shrink the window from the left by moving the left pointer until the distinct element count is valid.
4. Track whether the element Y is present within the window.
5. Whenever the window has exactly X distinct elements and contains Y, calculate the length of the current subarray and update the maximum length.

Code:

```
import java.util.*;

public class Q3 {
    public static void main(String args[]) {
        Scanner sc = new Scanner(System.in);

        long t = sc.nextLong();

        while (t-- > 0) {
            long n = sc.nextLong();
            long x = sc.nextLong();
            long y = sc.nextLong();

            long[] a = new long[(int) n];
            for (int i = 0; i < n; i++) {
                a[i] = sc.nextLong();
            }

            long l = 0, r = 0;
            HashMap<Long, Long> mp = new HashMap<>();
            long cnt = 0, maxi = 0;
            boolean flag = false;

            while (r < n) {
                mp.put(a[(int) r], mp.getOrDefault(a[(int) r], 0L) + 1);

                if (mp.get(a[(int) r]) == 1) {
                    cnt++;
                }

                while (cnt > x) {
                    mp.put(a[(int) l], mp.get(a[(int) l]) - 1);
                    if (mp.get(a[(int) l]) == 0) {
                        cnt--;
                    }
                    if (a[(int) l] == y && mp.get(a[(int) l]) == 0) {
                        flag = false;
                    }
                    l++;
                }

                if (a[(int) r] == y) {
                    flag = true;
                }

                if (cnt == x && flag) {
                    maxi = Math.max(maxi, r - l + 1);
                }
            }
        }
    }
}
```

```
        r++;
    }

    System.out.println(maxi);
}

sc.close();
}
```

Test Case 1:

```
2
3 2 1
2 3 1
1 1 2
1
```

Output:

```
2
0
```

Test Case 2:

```
1
5 2 5
5 1 5 1 5
```

Output:

```
5
```

Test Case 3:

```
3
7 2 4
4 4 1 2 3 4 4
10 4 1
1 2 3 4 5 1 2 3 1 1
6 3 3
3 1 3 2 4 3
```

Output:

```
3
6
4
```

Test Case 4:

```
2
6 2 1
1 1 2 3 4 1
5 3 7
7 7 7 7 1
```

Output:

```
3
0
```

4) The country of Byteland consists of n cities. Between any 2 cities it is possible to have a railway track and a road. Railway tracks are bidirectional, meaning if there exists a railway track between u and v then you can take a train from u to v as well as from v to u . Similarly, roads are bidirectional, meaning if there exists a route between u and v then you can drive from u to v as well as from v to u .

2 cities, u and v are called railway-connected if it is possible to travel between u and v using railway tracks.

2 cities, u and v are called road-connected if it is possible to travel between u and v using roads. The transportation network is called balanced if for all pairs of cities u, v :

u, v are railway-connected if and only if u, v are road-connected.

Initially, there are n cities and no roads or railways in Byteland. You will be given q instructions asking you to build either a railway track or a road between some 2 cities. After each instruction, you must report whether the transportation network is balanced.

Input format

The first line of input will contain 2 integers, n and q . q lines will follow. Each line will contain 3 space-separated integers in one of the following formats:

1 u v : build a railway track between u and v

2 u v : build a road between u and v

Output format

You must print q lines. The i th line contains an answer to the question whether the transport network is balanced after the i th instruction. If it is balanced print "YES" (without quotes) otherwise print "NO" (without quotes)

Algorithm:

1. Create two disjoint-set data structures (Union-Find) to manage railway and road connections separately.
2. For each query, either connect two cities with a railway or a road based on the instruction.
3. After each connection, check if the transportation network is balanced, meaning railway-connected cities should also be road-connected.
4. If the network is balanced, return "YES"; otherwise, return "NO".
5. Continue processing until all queries are handled

Code:

```
import java.util.*;

class UnionFind {
    private int[] parent, rank;

    public UnionFind(int n) {
        parent = new int[n];
        rank = new int[n];
        for (int i = 0; i < n; i++) {
            parent[i] = i;
            rank[i] = 0;
        }
    }
}
```

```
// Find function with path compression
public int find(int x) {
    if (parent[x] != x) {
        parent[x] = find(parent[x]);
    }
    return parent[x];
}

public void unionSets(int x, int y) {
    int rootX = find(x);
    int rootY = find(y);

    if (rootX != rootY) {
        if (rank[rootX] > rank[rootY]) {
            parent[rootY] = rootX;
        } else if (rank[rootX] < rank[rootY]) {
            parent[rootX] = rootY;
        } else {
            parent[rootY] = rootX;
            rank[rootX]++;
        }
    }
}

public boolean connected(int x, int y) {
    return find(x) == find(y);
}
}

public class Q4 {

    public static String checkBalance(int n, UnionFind railwayUnion, UnionFind
roadUnion) {
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {
                if (railwayUnion.connected(i, j) != roadUnion.connected(i, j)) {
                    return "NO";
                }
            }
        }
        return "YES";
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int q = sc.nextInt();
    }
}
```

```
UnionFind railwayUnion = new UnionFind(n);
UnionFind roadUnion = new UnionFind(n);

List<String> results = new ArrayList<>();

for (int i = 0; i < q; i++) {
    int t = sc.nextInt();
    int u = sc.nextInt() - 1;
    int v = sc.nextInt() - 1;

    if (t == 1) {
        railwayUnion.unionSets(u, v); // Railway union
    } else if (t == 2) {
        roadUnion.unionSets(u, v);    // Road union
    }

    results.add(checkBalance(n, railwayUnion, roadUnion));
}

for (String result : results) {
    System.out.println(result);
}

sc.close();
}
```

Input:

```
5 8
1 1 2
1 2 3
2 1 3
2 1 2
1 3 4
2 2 5
1 4 5
2 1 4
```

Output:

```
NO
NO
NO
YES
NO
NO
NO
YES
```